

Project_Definition_Document

Project Definition Document - dewit ToDo App

Einführung

Das **dewit** Projekt ist eine moderne, responsive ToDo-Webanwendung, die entwickelt wurde, um Studierenden und Berufstätigen eine einfache Möglichkeit zu bieten, ihre täglichen Aufgaben und universitären Abgabetermine zu organisieren. Die App ermöglicht es, Prioritäten zu setzen und Fristen einzuhalten, ohne Stress.

Ziele des Projekts

- Bereitstellung eines Minimum Viable Product (MVP) bis Juni/Juli 2026
- Features: Anlage von ToDos, Planung von Deadlines, Einstellen von Dringlichkeit (1-3), "Mein Tag"-Feature für Tagesplanung

Beteiligte Entwickler

- Robin Steininger
- Johannes Pernsteiner

Architekturübersicht

Die Anwendung folgt einer klassischen Client-Server-Architektur mit drei Hauptkomponenten:

1. **Frontend:** Vanilla JavaScript-Anwendung, serviert über Nginx
2. **Backend:** Express.js REST-API
3. **Datenbank:** PostgreSQL

```
Frontend (Nginx + Vanilla JS) ↔ Backend (Express.js) ↔ Datenbank (PostgreSQL)
```

Technologiestack

- **Frontend:** HTML5, CSS3, Vanilla JavaScript
- **Backend:** Node.js, Express.js, PostgreSQL (pg-Treiber)
- **Datenbank:** PostgreSQL
- **Deployment:** Docker, Docker Compose
- **Styling:** Responsive Design mit CSS-Variablen für Dark Mode

Dateien und ihre Aufgaben

Root-Verzeichnis

`README.md`

- Kurze Beschreibung des Projekts und des Repositories
- Zweck: Erste Orientierung für neue Entwickler

`SETUP.md`

- Detaillierte Anleitung zur Architektur und zum Setup
- Enthält Quick Start mit Docker, lokale Entwicklungsschritte, Features-Liste, Datenbankschema und API-Endpunkte
- Zweck: Vollständige Setup-Dokumentation

`Project Description - ToDo App - dewit!.md`

- Projektvision und Ziele
- Beschreibung der Features und Beteiligten
- Zweck: Überblick über die Projektziele und Roadmap

`Dewit_Backlog.mk`

- Akzeptanzkriterien für Features (z.B. Hinzufügen von Aufgaben)
- Zweck: Definition der Anforderungen und Qualitätskriterien

`index.html`

- Haupt-HTML-Datei der Anwendung
- Struktur: Header mit Titel und Dark Mode Toggle, "Mein Tag"-Sektion, Formular für neue Notizen, Such-/Filterleiste, Kalender, Notizen-Container
- Zweck: Grundgerüst der Benutzeroberfläche

`style.css`

- CSS-Stylesheet für das gesamte Styling
- Features: Responsive Design, CSS-Variablen für Farben, Dark Mode, Prioritätsfarben (wichtig=rot, mittel=orange, egal=grün)
- Zweck: Visuelle Gestaltung der Anwendung

`script-api.js`

- JavaScript-Datei für API-Kommunikation

- Funktionen: fetchNotes(), addNote(), deleteNote(), editNote(), toggleDone(), toggleMyDay()
- Zweck: Verbindung zwischen Frontend und Backend-API

script.js

- Alternative JavaScript-Implementierung mit localStorage
- Funktionen: addNote(), deleteNote(), editNote(), toggleDone(), toggleMyDay(), renderMyDay(), renderNotes(), renderCalendar()
- Zweck: Lokale Version ohne Backend (veraltet, wird durch script-api.js ersetzt)

nginx.conf

- Nginx-Konfigurationsdatei
- Features: Serviert statische Dateien, Proxy für API-Anfragen an Backend
- Zweck: Webserver-Konfiguration für Produktionsumgebung

Dockerfile

- Docker-Build-Datei für Frontend
- Multi-Stage Build: Node.js für Build, Nginx für Serving
- Zweck: Containerisierung des Frontends

docker-compose.yml

- Docker Compose-Datei für vollständige Anwendung
- Services: postgres, backend, frontend
- Zweck: Orchestrierung aller Komponenten für Entwicklung und Produktion

Backend-Verzeichnis (backend/)

README.md

- Spezifische Dokumentation für das Backend
- Zweck: Backend-spezifische Informationen

package.json

- Node.js-Projektdefinition
- Abhängigkeiten: express, pg (PostgreSQL-Treiber), cors, dotenv
- Dev-Abhängigkeiten: nodemon
- Scripts: start (Produktion), dev (Entwicklung)
- Zweck: Definition der Backend-Abhängigkeiten und Scripts

server.js

- Hauptserver-Datei mit Express.js
- Features: CORS-Unterstützung, JSON-Middleware, REST-API-Endpunkte
- Endpunkte:
 - GET /api/notes: Alle Notizen abrufen
 - POST /api/notes: Neue Notiz erstellen
 - PUT /api/notes/:id: Notiz aktualisieren
 - DELETE /api/notes/:id: Notiz löschen
- Zweck: Implementierung der REST-API

db.js

- Datenbankverbindungsmodul
- Verwendet pg-Pool für PostgreSQL-Verbindung
- Konfiguration über Umgebungsvariablen
- Zweck: Zentralisierte Datenbankverbindung

schema.sql

- SQL-Schema für die Datenbank
- Tabelle `notes` mit Feldern: `id`, `text`, `date`, `priority`, `done`, `in_my_day`, `created_at`, `updated_at`
- Indizes für `date` und `priority`
- Zweck: Definition der Datenbankstruktur

Dockerfile

- Docker-Build-Datei für Backend
- Node.js Alpine Image, Installation von Abhängigkeiten, Start des Servers
- Zweck: Containerisierung des Backends

docker-compose.yml

- Docker Compose für Backend-only Setup
- Services: `postgres`, `backend`
- Zweck: Unabhängiges Backend-Testing

Datenbankschema

```
CREATE TABLE notes (
  id SERIAL PRIMARY KEY,
  text VARCHAR(500) NOT NULL,
  date DATE NOT NULL,
  priority VARCHAR(20) NOT NULL CHECK (priority IN ('important', 'medium',
```

```
'low')),  
  done BOOLEAN DEFAULT FALSE,  
  in_my_day BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

API-Endpunkte

Methode	Endpoint	Beschreibung
GET	/api/notes	Alle Notizen abrufen
POST	/api/notes	Neue Notiz erstellen
PUT	/api/notes/:id	Notiz aktualisieren
DELETE	/api/notes/:id	Notiz löschen

Setup und Ausführung

Schnellstart mit Docker

```
docker-compose up
```

- Frontend: <http://localhost>
- Backend: <http://localhost:3001>
- Datenbank: Port 5432

Lokale Entwicklung

1. Backend: `cd backend && npm install && npm run dev`
2. Frontend: Öffne `index.html` im Browser oder verwende Live Server

Deployment

- Verwende Docker Compose für Produktion
- Für Portainer: Repository pushen und als Stack deployen

Funktionalitäten

-  ToDos erstellen, lesen, aktualisieren, löschen
-  Datumsbasierte Organisation
-  Prioritätsstufen (wichtig, mittel, egal)

- 🌟 "Mein Tag"-Sektion für heutige Aufgaben
- 🔍 Suche und Filter
- 🌙 Dark Mode
- 📱 Responsive Design

Dieses Dokument soll es ermöglichen, den Code des dewit-Projekts vollständig zu verstehen und weiterzuentwickeln.

c:\Users\JohannesPernsteiner\Documents\GitHub\dewit---ToDo-App\Project_Definition_Document.md